EECS 116/216 Project 2 - Team 2

Alex Aney (SID: 3037756490)
Sebastian Vargas (SID: 3040072402)
Thanos Kyriakopoulos (SID: 3043240840)

October 2026

1 Introduction

The purpose of this project is to estimate a quadrotor's pose and velocity from its downward facing camera. The camera is looking down at a grid of April tags (with some noise in the shape of a random scribble). In part 1 we use a planar homography and our knowledge of the April tag grid's geometry to recover the pose of the robot. In part 2 we use feature tracking, optical-flow velocity, and 3 point RANSAC to recover the velocity of the robot: the linear velocity of the body origin in the world frame and the angular velocity in the body frame.

Motion capture (mocap) measurements are used only to evaluate our estimates in the plots; they are never used as an input to either estimator, and neither are the recorded IMU readings.

2 Methodology

2.1 Frames and Conventions

We use three frames throughout this report:

- **World** $\{W\}$: origin at the top-left corner of the top-left tag, $+X$ down the mat, $+Y$ to the right, $+Z$ up from the floor. All tag corners lie in the plane $Z = 0$.
- **Body** $\{B\}$: the quadrotor body frame, which coincides with the IMU and mocap frames.
- **Camera** $\{C\}$: $+x$ right, $+y$ down, $+z$ forward along the optical axis (pointing at the floor).

We write T_{AB} for the homogeneous transform taking coordinates in frame B to coordinates in frame A, i.e. $\mathbf{p}_A = T_{AB}\mathbf{p}_B$. The supplied calibration is

$$\mathbf{p}_C = R_{CB}\mathbf{p}_B + \mathbf{t}_{CB}, \qquad R_{CB} = R_z(-\pi/4)R_x(\pi), \qquad \mathbf{t}_{CB} = \begin{bmatrix}-0.04 & 0 & -0.03\end{bmatrix}^\top \text{ m}, \tag{1}$$

which is `BODY_TO_CAMERA` $= T_{CB}$. Note that $\mathbf{t}_{CB}$ is the body origin expressed in camera coordinates, so the camera and body origins are offset by $\|\mathbf{t}_{CB}\| = 0.05$ m. The $R_x(\pi)$ factor flips the camera to look down, and $R_z(-\pi/4)$ accounts for the camera being mounted at 45° relative to the body x-axis. This offset matters for both parts: the body position is not the camera position, and when the body rotates the camera translates, so both the rotation and the translation of T_{CB} must be applied. Pixel coordinates (u, v) are zero-based with u right and v down; images and tag detections are already rectified, so the supplied intrinsic matrix K is the only calibration we apply.

2.2 Homography Based Pose Estimation

2.2.1 Getting the Corners

We are told that each tag is a 0.152 m square, and the gap between each tag is 0.152 m, except for between columns 3 and 4 and between columns 6 and 7, where it's 0.178 m. We are also informed that the image is in the Z = 0 plane.

For each id, its row and column are

$$r = \text{id} \bmod 12, c = \left\lfloor \frac{\text{id}}{12} \right\rfloor, s = 0.152, \tag{2}$$

$$\delta(c) = 0.026\,\mathbf{1}_{c>2} + 0.026\,\mathbf{1}_{c>5}, \qquad x = (2r+1)s, \qquad y = 2cs + \delta(c). \tag{3}$$

Here (x, y) is the bottom-left corner p_1 of the tag. Because $+X$ points down the mat, the bottom edge of a tag has the larger X value, which is why x includes the extra $+s$. The extra 0.026 m $(= 0.178 - 0.152)$ is added once for

every wide gap to the left of column c. The remaining corners follow the required order (bottom left, bottom right, top right, top left):

$$p_1 = (x,\, y,\, 0), \quad p_2 = (x,\, y+s,\, 0), \quad p_3 = (x-s,\, y+s,\, 0), \quad p_4 = (x-s,\, y,\, 0). \tag{4}$$

The function returns these in the same order as the input IDs, so world corner k of tag j is paired with detected pixel corner k of tag j.

2.2.2 Recovering the Homography

With our corners, we are going to use a planar homography to estimate our pose. We define our homography as follows:

$$\mathbf{x}_i = \begin{bmatrix} x_i \\ y_i \\ 1 \end{bmatrix}, \qquad \mathbf{x}'_i = \begin{bmatrix} x'_i \\ y'_i \\ 1 \end{bmatrix}, \qquad \mathbf{x}'_i \sim H\mathbf{x}_i \tag{5}$$

We then reformat our equation such that our H is instead a vector in $\mathbb{R}^9$. This gives us a new equation, $A\mathbf{h} = \mathbf{0}$ defined by the following A matrix:

$$A_i = \begin{bmatrix} x_i & y_i & 1 & 0 & 0 & 0 & -x'_i x_i & -x'_i y_i & -x'_i \\ 0 & 0 & 0 & x_i & y_i & 1 & -y'_i x_i & -y'_i y_i & -y'_i \end{bmatrix} \qquad A = \begin{bmatrix} A_1 \\ A_2 \\ \vdots \\ A_N \end{bmatrix}, \tag{6}$$

These two rows come from eliminating the unknown scale: writing $\mathbf{h}_k^\top$ for the k-th row of H, the projection gives $x'_i = \mathbf{h}_1^\top \mathbf{x}_i / \mathbf{h}_3^\top \mathbf{x}_i$ and $y'_i = \mathbf{h}_2^\top \mathbf{x}_i / \mathbf{h}_3^\top \mathbf{x}_i$; multiplying through by the denominator gives two equations that are linear in $\mathbf{h}$. Each visible tag contributes four corners, i.e. eight rows, and we stack the rows of every visible tag into a single system. Here (x'_i, y'_i) are the raw pixel coordinates (u_i, v_i) of the detected corners; K is removed afterwards.

In order to recover h, we take the SVD of our A matrix. We know that our h must lie in the null space of A. Since our equation is homogeneous, we will only be able to recover h up to a scalar factor. This means we can arbitrarily decide our scaling, and our H only has 8 degrees of freedom. Therefore, our last column of our V matrix is the null space of our A matrix, and is an acceptable solution for $\mathbf{h}$.

$$A = U\Sigma V^\top, \mathbf{h} = V_{:,9} \qquad H = \begin{bmatrix} h_1 & h_2 & h_3 \\ h_4 & h_5 & h_6 \\ h_7 & h_8 & h_9 \end{bmatrix} \tag{7}$$

With a single tag A is only 8×9, so we keep NumPy's default `full_matrices=True`, which still returns all nine right singular vectors including the null-space one.

Because we can only recover the homography up to a scale factor, we should test the sign convention. Fortunately, we can take an arbitrary point, say one of our corners, and apply the homography. If the output has a z value less than 0, we know that our homography is mapping to coordinates behind the camera, so we can flip the sign of our homography matrix H. This works because the third component of $H\mathbf{x}_i$ is proportional to the scale factor λ below, which is the camera depth of the point and must be positive. In our implementation the test point is the first detected corner.

Then, we look to our standard transformation from world coordinates to image coordinates. Breaking it down, we see that for images at the plane Z = 0, we can ignore the column $\mathbf{r}_3$.

$$\lambda \begin{bmatrix} u \\ v \\ 1 \end{bmatrix} = K \begin{bmatrix} \mathbf{r}_1 & \mathbf{r}_2 & \mathbf{r}_3 & \mathbf{t}_{CW} \end{bmatrix} \begin{bmatrix} X \\ Y \\ 0 \\ 1 \end{bmatrix} = K \begin{bmatrix} \mathbf{r}_1 & \mathbf{r}_2 & \mathbf{t}_{CW} \end{bmatrix} \begin{bmatrix} X \\ Y \\ 1 \end{bmatrix}, \qquad H \sim K \begin{bmatrix} \mathbf{r}_1 & \mathbf{r}_2 & \mathbf{t}_{CW} \end{bmatrix} \tag{8}$$

In this form, we can see that a homography is in the same shape as our transformation ignoring the z column corrected by our camera intrinsics matrix. Therefore, to recover our original transformation, we take:

$$G = K^{-1}H = \begin{bmatrix} \mathbf{g}_1 & \mathbf{g}_2 & \mathbf{g}_3 \end{bmatrix} \sim \begin{bmatrix} \mathbf{r}_1 & \mathbf{r}_2 & \mathbf{t}_{CW} \end{bmatrix}, \tag{9}$$

$$\mathbf{t}_{CW} = \frac{\mathbf{g}_3}{\|\mathbf{g}_1\|}, \qquad \tilde{R}_{CW} = \begin{bmatrix} \mathbf{g}_1 & \mathbf{g}_2 & \mathbf{g}_1 \times \mathbf{g}_2 \end{bmatrix}, \qquad R_{CW} = \mathrm{proj}_{SO(3)} \left(\tilde{R}_{CW} \right) \tag{10}$$

Since $\mathbf{r}_1$ is a unit vector, $\|\mathbf{g}_1\|$ is the unknown scale of H, so dividing $\mathbf{g}_3$ by it recovers the metric translation. The third rotation column is the cross product of the first two, because the columns of a rotation matrix are orthonormal and right-handed. With noisy corners, $\mathbf{g}_1$ and $\mathbf{g}_2$ are generally neither unit length nor exactly orthogonal, so $\tilde{R}_{CW}$ is only approximately a scaled rotation. We therefore convert it into a valid rotation using SciPy's `Rotation.from_matrix`, which always returns a proper rotation ($\det R_{CW} = +1$) rather than a reflection. Together with the positive-depth sign choice, this removes the two-fold ambiguity in the decomposition. Because the columns of $\tilde{R}_{CW}$ are not individually normalized before this step, this projection is an approximation.

Our camera pose in the world frame is the same as the transformation below, which we can recover from our newly found world to camera transformation and the given `BODY_TO_CAMERA` transformation.

$$T_{CW} = \begin{bmatrix} R_{CW} & \mathbf{t}_{CW} \\ \mathbf{0}^\top & 1 \end{bmatrix}, \quad T_{WB} = T_{CW}^{-1} T_{CB} = \begin{bmatrix} R_{WB} & \mathbf{t}_{WB} \\ \mathbf{0}^\top & 1 \end{bmatrix} \tag{11}$$

Reading right to left, T_{CB} takes a body-frame point into the camera frame and $T_{CW}^{-1} = T_{WC}$ takes it from the camera frame into the world frame. Applying the full homogeneous T_{CB} (not only its rotation) is what moves the position estimate from the camera origin to the body origin. The returned pose is $(\mathbf{t}_{WB}, R_{WB})$.

If no tags are detected in a frame, the system has no rows and the pose is unobservable, so `estimate_pose` returns `None` and the plot leaves a gap. For plotting, the runner converts R_{WB} to roll, pitch and yaw using the intrinsic ZYX convention $R_{WB} = R_z(\text{yaw}) R_y(\text{pitch}) R_x(\text{roll})$, matching the mocap data.

2.3 Corner Extraction and Tracking

2.3.1 Feature Extraction

For this part of the assignment, we were tasked with extracting feature correspondences from the image without knowledge of how the AprilTag grid was organized, given two consecutive frames and our drone's pose in the world frame. To do this, we used OpenCV's `goodFeaturesToTrack` and `calcOpticalFlowPyrLK` methods. We first called `goodFeaturesToTrack` to identify features in our first image, then used `calcOpticalFlowPyrLK` in order to find where those features are in the second image. This gave us a set of correspondences we could use, x and x'. Before we could calculate the flow, we had to normalize our pixels, using our camera intrinsics matrix and our timestep Δt.

Features are detected on the whole previous image, so candidates come from both the tag corners and the random scribble. Points that `calcOpticalFlowPyrLK` reports as failed (`status` $= 0$) are discarded. We assume every retained feature lies on the ground plane, which holds for the mat and scribble. If fewer than three corners are detected, fewer than three tracks succeed, or $\Delta t \leq 0$, the frame pair is skipped (`None`).

Detector and tracker parameters. Table 1 lists the parameters used in `estimate_velocity` and the reasoning behind each. Parameters marked † are not set explicitly and use OpenCV's defaults.

Parameter	Value	Reason
`maxCorners`	100	Enough points for the 6-unknown fit while keeping runtime low
`qualityLevel`	0.5	Keeps only corners at least half as strong as the strongest one, favouring sharp, high-contrast corners that are easier to track
`minDistance`	10 px	Spreads features over the image, which improves conditioning of H
`blockSize`	$3^\dagger$	Shi–Tomasi corner response window
LK `winSize`	$21 \times 21^\dagger$	Search window for each tracked point
LK `maxLevel`	$3^\dagger$	Pyramid levels to handle larger displacements during fast motion
LK termination	30 it. or $\epsilon = 0.01^\dagger$	Standard accuracy/time trade-off

Table 1: Feature detection and KLT tracking parameters used in `flow.py`. †OpenCV default (not set explicitly).

$$\dot{\mathbf{x}}_i \approx \frac{1}{\Delta t} \left[K^{-1} \left(\mathbf{x}'_{i,t+\Delta t} - \mathbf{x}'_{i,t} \right) \right] \tag{12}$$

Here Δt is the difference between the current and previous image timestamps. Because the third component of a pixel difference is zero, only the first two components of K^{-1} act on it, so normalized flow is simply the pixel displacement divided by the focal lengths, $\dot{x}_i \approx \Delta u_i / (f_x \Delta t)$, $\dot{y}_i \approx \Delta v_i / (f_y \Delta t)$, in units of s^{-1}.

Then, using our body to world frame transformation and our body to camera transformation, we create a camera to world frame transformation. Now what we need is our image depths for each point, which we can calculate using this transformation. We first take our points and represent them in the world frame using our transformation. Then,

we need to calculate our depth as the distance along our point ray such that it intersects Z = 0. We do so with the following:

$$T_{WC} = T_{WB}T_{BC} = T_{WB}T_{CB}^{-1} = \begin{bmatrix} R_{WC} & \mathbf{p}_{WC} \\ \mathbf{0}^\top & 1 \end{bmatrix}, \tag{13}$$

$$\mathbf{d}_{W,i} = R_{WC}\mathbf{d}_{C,i}, \quad \mathbf{d}_{C,i} = K^{-1}\mathbf{x}'_{i,t} = \begin{bmatrix} x_i & y_i & 1 \end{bmatrix}^\top, \quad \mathbf{X}_{W,i} = \mathbf{p}_{WC} + Z_i\mathbf{d}_{W,i} \tag{14}$$

The above represents our image feature in the world frame as sum of our translation from the world frame to the camera frame and the direction of our points ray in the world frame, $\mathbf{d}_{W,i}$ with some unknown depth along that ray Z_i. In order to know when this intersects with $Z = 0$, we need Z_i such that:

$$0 = p_{WC,z} + Z_i d_{W,i,z} \quad \Longrightarrow \quad Z_i = -\frac{p_{WC,z}}{d_{W,i,z}}, \text{ s.t. } Z_i > 0 \tag{15}$$

Because $\mathbf{d}_{C,i}$ has its third component equal to 1 (it is not a unit vector), the scalar Z_i is exactly the camera-z coordinate of the point, $\mathbf{X}_{C,i} = Z_i\mathbf{d}_{C,i}$. This is the depth that appears in the image-motion equations below. It is neither the body height nor the Euclidean distance along the ray; using the ray length would overestimate depth for features near the image edge. We use the previous frame's pose and pixel positions here, since the flow is measured starting from the previous frame. Points with non-positive or non-finite depth (rays parallel to or pointing away from the floor), or with non-finite coordinates or flow, are discarded, and the frame pair is skipped if fewer than three remain.

With all of this we are ready to fit our velocity and our angular velocity to our optical flow.

2.3.2 Velocity Estimation

To estimate our flow, we begin with the following:

$$\dot{x}_i = \frac{\dot{X}_i Z_i - X_i \dot{Z}_i}{Z_i^2} = \frac{\dot{X}_i - x_i \dot{Z}_i}{Z_i}, \;\; \dot{y}_i = \frac{\dot{Y}_i Z_i - Y_i \dot{Z}_i}{Z_i^2} = \frac{\dot{Y}_i - y_i \dot{Z}_i}{Z_i} \tag{16}$$

This is the quotient rule applied to the normalized projection $x_i = X_i/Z_i$, $y_i = Y_i/Z_i$, where (X_i, Y_i, Z_i) are the camera-frame coordinates of feature i.

And we know that, for a point, its velocity is defined as:

$$\dot{\mathbf{X}}_i = -\mathbf{v}_C - \boldsymbol{\omega}_C \times \mathbf{X}_i \tag{17}$$

This holds because the feature is static in the world: in the moving camera frame it appears to move opposite to the camera's own translation $\mathbf{v}_C$ (velocity of the camera origin, expressed in the camera frame) and rotation $\boldsymbol{\omega}_C$ (camera angular velocity, expressed in the camera frame).

Using our definitions for our flow and plugging in our values we've defined above, we get the following two equations:

$$\dot{x}_i = \frac{-v_x + x_i v_z}{Z_i} + x_i y_i \omega_x - (1 + x_i^2)\omega_y + y_i \omega_z, \qquad \dot{y}_i = \frac{-v_y + y_i v_z}{Z_i} + (1 + y_i^2)\omega_x - x_i y_i \omega_y - x_i \omega_z \tag{18}$$

$$\boxed{\dot{\mathbf{x}}_i = \frac{1}{Z_i} A(\mathbf{x}_i)\mathbf{v}_C + B(\mathbf{x}_i)\boldsymbol{\omega}_C} \tag{19}$$

Equivalently, in the form requested by the assignment, each feature contributes two 1×6 rows f_1, f_2:

$$f_1(x, y, Z) = \begin{bmatrix} -\frac{1}{Z} & 0 & \frac{x}{Z} & xy & -(1 + x^2) & y \end{bmatrix}, \qquad f_2(x, y, Z) = \begin{bmatrix} 0 & -\frac{1}{Z} & \frac{y}{Z} & 1 + y^2 & -xy & -x \end{bmatrix}, \tag{20}$$

acting on $\begin{bmatrix} V_x & V_y & V_z & \Omega_x & \Omega_y & \Omega_z \end{bmatrix}^\top$. $A(\mathbf{x}_i)/Z_i$ and $B(\mathbf{x}_i)$ in the boxed equation are the first three and last three columns of $\begin{bmatrix} f_1 \\ f_2 \end{bmatrix}$. Two observations explain why depth is required: the translational part is scaled by $1/Z_i$, so without depth we could only recover $\mathbf{v}_C$ up to scale; the rotational part does not depend on depth at all. In `flow.py`, the helper `create_A_B` fills the two $2N \times 3$ blocks $A(\mathbf{x})/Z$ and $B(\mathbf{x})$ row by row from these expressions.

We can define $H_i = \begin{bmatrix} \frac{1}{Z_i} A(\mathbf{x}_i) & B(\mathbf{x}_i) \end{bmatrix}$ and then stack each matrix vertically to give us a matrix H, such that $\dot{\mathbf{x}} = H\mathbf{V}$. Due to noise and inexact measurements, we cannot get an exact solution to this to solve for our V. We define our problem of the form:

$$\arg\min_{\mathbf{V}^*} ||H\mathbf{V}^* - \dot{\mathbf{x}}||_2^2 \tag{21}$$

Here $H \in \mathbb{R}^{2N\times 6}$ and $\dot{\mathbf{x}} \in \mathbb{R}^{2N}$ interleaves $\dot{x}_1, \dot{y}_1, \dot{x}_2, \dot{y}_2, \ldots$ so that its rows line up with those of H. Each feature gives two equations and there are six unknowns, so at least three features are needed; with fewer than three usable tracks we return `None`.

2.3.3 Least Squares

We can fit all of our points for the above problem using our least squares solution:

$$\mathbf{V}^* = H^\dagger \dot{\mathbf{x}} \tag{22}$$

This gives us a velocity estimate that fits all points, including any noisy points or outliers from our feature set, which can work for a really good set of feature points, but is not robust. We compute $H^\dagger$ with `np.linalg.pinv`.

2.3.4 RANSAC

We can instead try to fit three points (the minimum needed as we have 6 unknowns in our equation) for our velocity and choose the motion that agrees with the most points. Note that we throw away any solution that is rank deficient. We do so using RANdom SAmple Consensus (RANSAC). For RANSAC we choose two parameters, the number of iterations k and our threshold t. Our knowns are our points, our flow, and our depths. Then, for k iterations, we build our H as we did previously using only three randomly selected points, call it H_S. We then fit these points with least squares:

$$\mathbf{V_S} = H_S^\dagger \dot{\mathbf{x}_S} \tag{23}$$

The three points are sampled without replacement so they are distinct. H_S is 6×6; if its rank (computed with `np.linalg.matrix_rank`) is below 6, the system does not determine a unique motion, so we skip that sample instead of accepting an arbitrary pseudo-inverse solution.

We then count how many inliers we have using the 2D flow residual of every feature $i = 1, \ldots, N$ (not only the three sampled points, which are fit exactly):

$$r_i = \|H_i \mathbf{V_S} - \dot{\mathbf{x}}_i\|_2 < t, \qquad \mathcal{I}_S = \{\, i : r_i < t \,\} \tag{24}$$

Here H_i is the 2×6 block of feature i and r_i is the norm of its two-component optical-flow residual, in s^{-1}.

After performing all k iterations (our implementation does not stop early), we take the largest set of inliers that we got for a certain velocity as our inlier group; on ties the first such set is kept. We then finally get our velocity estimate as:

$$\mathbf{V}^* = H_{inliers}^\dagger \dot{\mathbf{x}_{\mathbf{inliers}}} \tag{25}$$

The final refit over all inliers matters because a three-point solution is fit to only six equations and is therefore noisy; refitting over the full consensus set gives a much more accurate estimate while still excluding outliers. If every sample is rejected as rank deficient or no sample produces any inliers, we fall back to the least-squares fit over all points.

Choosing the number of iterations. If a fraction w of features are inliers, a random three-point sample is all inliers with probability w^3. To draw at least one all-inlier sample with probability p, we need

$$k \geq \frac{\ln(1-p)}{\ln(1-w^3)}. \tag{26}$$

We use $k = 15$. With $p = 0.99$ this is sufficient whenever $w \geq 0.64$; even for $w = 0.5$ the probability of drawing at least one clean sample is still $1-(1-0.5^3)^{15} \approx 0.87$ (reaching 0.99 at $w = 0.5$ would need $k \geq 35$). We chose a small k to keep runtime low, since each iteration solves a 6×6 system and scores every feature, and RANSAC runs for every frame pair.

Choosing the threshold. We use $t = 0.1\ \mathrm{s}^{-1}$. The threshold is in normalized flow units, so it corresponds to a pixel error of $t\, f\, \Delta t \approx 0.1 \times 311 \times \Delta t \approx 31\, \Delta t$ pixels accumulated over one frame interval. The aim is a threshold a few times larger than the error of a correctly tracked feature, so those features are kept, while mistracked features with much larger errors are rejected.

2.3.5 Finding World Velocity

Given our estimate $\mathbf{V}^* = \begin{bmatrix} \mathbf{v}_C \\ \boldsymbol{\omega}_C \end{bmatrix}$, we then represent it in the world frame using:

$$\boldsymbol{\omega}_B = R_{BC}\boldsymbol{\omega}_C, \qquad \mathbf{v}_W = R_{WC}\mathbf{v}_C + R_{WB}\left(\boldsymbol{\omega}_B \times R_{BC}\mathbf{p}_{CB}\right) \tag{27}$$

Derivation. The camera and body are one rigid object, so they share the same angular velocity; only the frame it is expressed in changes, which gives $\boldsymbol{\omega}_B = R_{BC}\boldsymbol{\omega}_C$ with $R_{BC} = R_{CB}^{\top}$. For the linear velocity, the body origin in world coordinates is $\mathbf{p}_{WB} = \mathbf{p}_{WC} + R_{WC}\,\mathbf{p}_{CB}$, where $\mathbf{p}_{CB} = \mathbf{t}_{CB}$ is the (constant) body origin in camera coordinates. Differentiating, and using $\dot{R}_{WC} = R_{WC}[\boldsymbol{\omega}_C]_\times$,

$$\dot{\mathbf{p}}_{WB} = R_{WC}\mathbf{v}_C + R_{WC}\left(\boldsymbol{\omega}_C \times \mathbf{p}_{CB}\right). \tag{28}$$

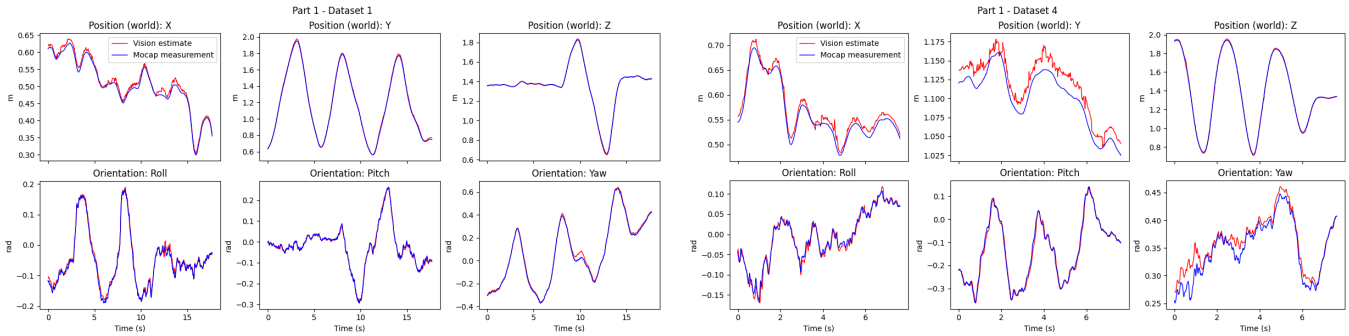
The second term is the lever-arm effect: when the body rotates, the offset camera moves even if the body origin does not, so this term subtracts out the part of the camera's velocity that is caused purely by rotation. Re-expressing the cross product in the body frame ($R_{WC} = R_{WB}R_{BC}$, and rotations preserve cross products) gives exactly the expression above, which is how `estimate_velocity` computes it. Since $\|\mathbf{p}_{CB}\| = 0.05$ m, this term has magnitude up to $0.05\,\|\boldsymbol{\omega}\|$ m/s, e.g. 5 cm/s at 1 rad/s, which is not negligible compared to the measured linear velocities.

As instructed, all rotations here (R_{WC}, R_{WB}) come from the pose of the previous frame. The estimate is an average over the interval $[t_{k-1}, t_k]$, so the runner plots it at the interval midpoint $(t_{k-1} + t_k)/2$.

Libraries. We implemented the homography estimation (building A and solving it by SVD), its decomposition (sign choice, removing K, recovering the scale, translation and rotation columns), the least-squares fit, and RANSAC ourselves with NumPy (`np.linalg.svd`, `pinv`, `matrix_rank`). The only library routines are OpenCV's corner detector and KLT tracker, and SciPy's `Rotation.from_matrix` for the final projection of our recovered rotation matrix onto $SO(3)$.

3 Results

3.1 Part 1: Vision Based Pose Estimation



(a) Dataset 1 (b) Dataset 4

Figure 1: Part 1 pose: vision estimates (red) and mocap measurements (blue).

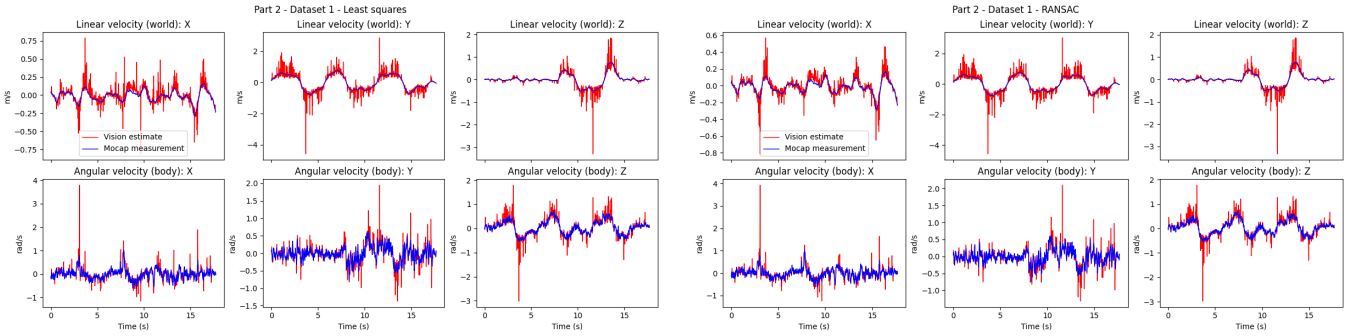
Agreement with mocap. Figure 1 shows that the homography-based pose tracks the mocap measurements closely in all six degrees of freedom on both datasets. In Dataset 1 (about 18 s), the large Y sweeps between 0.6 and 2.0 m and the Z excursion between 0.65 and 1.85 m are almost indistinguishable from mocap, and X stays within about 1–2 cm. Roll and pitch (within ± 0.3 rad) follow mocap closely, and yaw only overshoots slightly around 10 s. In Dataset 4 (about 7.5 s), Z oscillates between 0.75 and 1.95 m and again matches almost exactly, and roll and pitch track well. The largest visible errors are in Dataset 4's Y and yaw. Y only moves over about 15 cm, so the plot is zoomed in, and the estimate sits about 1–2 cm above mocap for most of the run. Yaw is noisier by about 0.02–0.03 rad during the first 2 s. These errors are small in absolute terms; they stand out because those axes barely move. Because the pose is computed independently for every frame, errors do not accumulate over time.

Sources of error. The remaining discrepancies come mainly from two sources:

- **Corner noise.** Detection errors of a fraction of a pixel are amplified into pose errors, especially when few tags are visible or they cover a small part of the image (e.g. at higher altitude or near the edge of the mat).

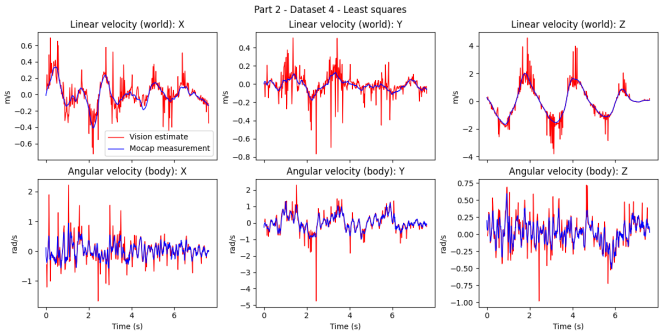
- **Calibration and timing.** Any error in the provided K, T_{CB}, the tag map, or the alignment between image and mocap timestamps appears as a bias or lag.

3.2 Part 2: Corner Extraction and Tracking

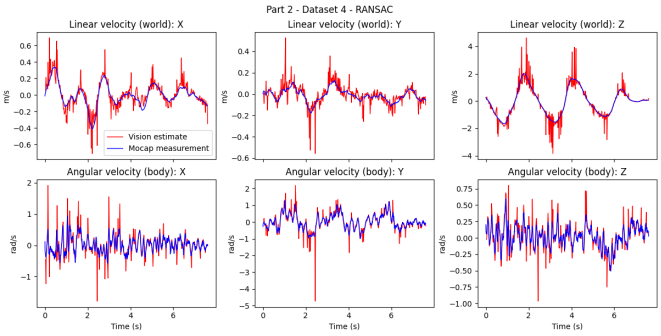


(a) Dataset 1, least squares

(b) Dataset 1, RANSAC



(c) Dataset 4, least squares



(d) Dataset 4, RANSAC

Figure 2: Part 2 velocity: vision estimates (red) and mocap measurements (blue). Top rows of each panel: world-frame linear velocity; bottom rows: body-frame angular velocity.

Agreement with mocap. With RANSAC enabled (Figures 2b and 2d), the vision estimates follow the overall shape of the mocap linear and angular velocities on both datasets. The large, slow motions are tracked best: the ± 2 m/s swings of v_y in Dataset 1 and of v_z in Dataset 4, the vertical maneuver in v_z between 10 and 15 s in Dataset 1, and the ± 1 rad/s swings of ω_z in Dataset 1. The smaller components, such as v_x (within about ± 0.6 m/s) and ω_x, ω_y in Dataset 4, follow the trend but sit closer to the noise level. On top of this, the estimates show two kinds of error: steady high-frequency noise of roughly ± 0.1–0.3 m/s around the mocap curve, and isolated large spikes. Examples of the spikes are v_y dropping to about -4 m/s near 4 s and v_z to about -3 m/s near 10 s in Dataset 1, and ω_y reaching about -4.5 rad/s near 2.5 s in Dataset 4. The velocity estimates are noisier than the Part 1 pose estimates. This is expected: each estimate is a finite difference over a single short frame interval, so small pixel tracking errors are divided by a small Δt.

Errors in the Part 1 pose also propagate into Part 2 through the depths. The translational flow scales as $\mathbf{v}_C/Z_i$, so a relative depth error $\delta Z/Z$ causes the same relative error in $\mathbf{v}_C$, while $\boldsymbol{\omega}_C$ is unaffected because the rotational flow does not depend on depth.

Effect of outlier rejection. Comparing the least-squares fits (Figures 2a and 2c) with the RANSAC fits, RANSAC gives a modest improvement. Some spikes are reduced: the largest positive v_x spike in Dataset 1 drops from about 0.7 to about 0.5 m/s, and the most negative v_y values in Dataset 4 shrink from about -0.75 to about -0.6 m/s. However, most of the large spikes appear at the same times and with similar sizes in both versions, for example v_y near 4 s, v_z near 10 s and ω_x near 3 s in Dataset 1, and ω_y near 2.5 s and v_z near 2 s in Dataset 4. Between spikes, the two methods are nearly identical. Least squares weights each residual quadratically, so a few mistracked features can pull the whole fit; RANSAC excludes features that disagree with the consensus motion, which removes this effect. However, RANSAC cannot remove errors that affect all features in the same way, such as an error in Δt or in the Part 1 pose used for the depths, because those features still agree with each other. In addition, our high `qualityLevel` keeps only strong corners, and when there are no gross outliers the RANSAC refit is essentially the least-squares solution.

Effect of timestamp jitter. The flow is computed as $\dot{\mathbf{x}}_i \approx \Delta\mathbf{x}_i/\Delta t$. If the recorded interval is $\Delta t + \delta t$ while the true interval is Δt, every flow vector is scaled by the same factor $\Delta t/(\Delta t+\delta t)$. Because the motion model is linear in $\mathbf{V}$, the estimated linear and angular velocities are scaled by this same factor, giving a relative error of approximately $-\delta t/\Delta t$. Two consequences follow:

- The error is multiplicative, so it is largest when the quadrotor moves fastest; at rest jitter has almost no effect.
- Because Δt is short, even a few milliseconds of timestamp error is a significant relative error, and it affects every channel of that frame's estimate at once.

Because the error is common to all features in a frame, RANSAC cannot reject it. Temporal smoothing over several frames, or fusing with the IMU, would reduce this noise, but we report per-interval estimates without IMU inputs as the assignment requires.

4 Conclusion

We estimated the quadrotor's full six-degree-of-freedom pose from AprilTag detections using an SVD-based planar homography and its decomposition with the camera intrinsics, followed by a rigid-body transform from the camera to the body frame, and the body velocity from KLT optical flow with three-point RANSAC. The pose estimates closely follow the mocap measurements. The velocity estimates follow the overall motion but are noisier, since they are finite differences over short frame intervals, and RANSAC reduces only the errors caused by individual mistracked features.